

Лабораторная работа №22

Изучение механизмов синхронизации потоков и обмена данными между ними

1 Цель работы

1.1 Научиться разрабатывать многопоточные приложения на C#;

1.2 Научиться создавать и применять фоновые и основные потоки и выполнять обмен данных между ними в программах на C#.

2 Литература

2.1 Фленов, М. Е. Библия C#. 4 изд / М. Е. Фленов. – Санкт-Петербург: БХВПетербург, 2019. – 512 с. – URL: <https://ibooks.ru/bookshelf/366634/reading>. – Режим доступа: только для зарегистрированных пользователей. – Текст : электронный. – гл.15.

3 Подготовка к работе

3.1 Повторить теоретический материал (см. п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Создание и приостановка потоков

Создать консольное приложение, где в основном потоке Main создается новый поток, исполняющий метод Print, который непрерывно печатает символ 1 в бесконечном цикле. Одновременно главный поток непрерывно печатает символ 0 в бесконечном цикле. В конце каждого шага цикла должна выполняться задержка:

Thread.Sleep(1000);

Вместо 1000 в разных циклах указать разные значения — поток будет приостанавливаться на указанное количество миллисекунд.

5.2 Обмен данными между потоками

Создать консольное приложение, в котором происходит обмен данными между потоками Main() и MyThread() через статическое поле (глобальную переменную) commonVar.

Второй поток исполняется (выводит некоторое сообщение) до того момента, пока первый поток не изменит переменную str на значение «х».

5.3 Обмен данными через канал

5.3.1 Создать консольное приложение, реализующее обмен данными между потоками по схеме, один производитель — несколько потребителей с использованием канала.

5.3.2 Поток Producer (производитель) генерирует последовательность чисел (например, от 1 до 20) и записывает их в канал с некоторой задержкой.

5.3.3 Три потока Consumer (потребители) читают данные из канала и

каждый обрабатывает полученные значения (выводит в консоль с указанием ID потребителя)

5.4 Синхронизация потоков

5.4.1 Создать консольное приложение, в котором 5 потоков одновременно увеличивают общий счётчик counter.

5.4.2 Каждый поток выполняет цикл (1000 итераций с небольшой задержкой), в котором увеличивает counter.

5.4.3 Выполнить все потоки, сравнить полученный результат с ожидаемым

5.4.4 Добавить в приложение синхронизацию при помощи lock, после чего повторно выполнить все потоки и сравнить результат с ожидаемым.

5.5 Отмена выполнения потоков

5.5.1 Создать консольное приложение, в котором реализовать метод имитирующий длительную загрузку файлов и выводящее сообщение с процессом на экран каждую секунду.

5.5.2 Реализовать отмену загрузки при вводе в процессе загрузки команды «С» при помощи CancellationToken.

6 Порядок выполнения работы

6.1 Запустить MS Visual Studio и создать оконное приложение C#.

6.2 Выполнить все задания из п.5 в одном решении.

6.3 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 В чем отличие потока от процесса?

8.2 Какие преимущества дает многопоточная архитектура?

8.3 Какие существуют основные средства синхронизации потоков?

8.4 Каким образом на однопроцессорных компьютерах исполняются многопоточные приложения?

8.5 Для чего в C# используется класс Thread?